

Sprawozdanie z projektu

Zaawansowane programowanie PLC

Temat: System sterowania automatycznym zbiornikiem
mieszającym

Autor:

inż. Maciej Lutyński

Numer albumu: 24289

Grupa laboratoryjna: 212MCAI_B

July 10, 2026

Contents

Chapter 1

Wstęp i cel projektu

Celem projektu było opracowanie oraz uruchomienie w środowisku Siemens TIA Portal programu sterującego procesem technologicznym, który w sposób kompleksowy wykorzystuje podstawowe konstrukcje programistyczne stosowane w automatyce przemysłowej. Program napisano w języku strukturalnym SCL (ang. *Structured Control Language*), zgodnym ze standardem IEC 61131-3.

Projekt miał spełniać następujące wytyczne programowe:

1. Funkcje logiczne (AND, OR, NOT),
2. Funkcje arytmetyczne,
3. Instrukcje warunkowe (IF, CASE),
4. Pętle (FOR),
5. Timery (TON),
6. Liczniki (CTU, CTD),
7. Sterowanie zaworem analogowym,
8. Detekcję zboczy sygnałów (R_TRIG, F_TRIG).

Jako obiekt sterowania wybrano zbiornik technologiczny z automatycznym dozowaniem cieczy, mieszaniem oraz spustem — proces typowy dla przemysłu chemicznego, spożywczego i farmaceutycznego, w którym w naturalny sposób występują wszystkie wymienione wyżej elementy programistyczne.

Chapter 2

Opis procesu technologicznego

Sterowany obiekt to zbiornik wyposażony w:

- **czujnik poziomu analogowy** — sygnał wejściowy 0–27648 (odpowiadający zakresowi 0–100% wypełnienia zbiornika),
- **zawór dozujący sterowany analogowo** — sygnał wyjściowy AO 0–27648, sterowany proporcjonalnie do uchybu regulacji,
- **pompę mieszającą** — sygnał dwustanowy (ON/OFF),
- **zawór spustowy** — sygnał dwustanowy (ON/OFF),
- wejścia operatorskie: **Start**, **Stop**, **Kasuj_Alarm**,
- wejście awarii zewnętrznej (np. zabezpieczenie termiczne silnika mieszadła).

Cykl pracy zbiornika przebiega w czterech głównych etapach, zaimplementowanych jako maszyna stanów:

1. **IDLE** (stan 0) — oczekiwanie na sygnał start przy spełnionych warunkach zezwolenia,
2. **NAPEŁNIANIE** (stan 1) — otwarcie zaworu dozującego proporcjonalnie do różnicy poziomu zadanego i rzeczywistego, aż do osiągnięcia poziomu zadanego,
3. **MIESZANIE** (stan 2) — załączenie pompy mieszającej na czas zadany przez timer,
4. **SPUST** (stan 3) — otwarcie zaworu spustowego na zadany czas,
5. **ZAKOŃCZONO** (stan 4) — inkrementacja licznika wykonanych cykli (batchy) i powrót do stanu IDLE.

Dodatkowo program rejestruje historię 10 ostatnich pomiarów poziomu i wylicza z nich wartość średnią, a każda zarejestrowana awaria zmniejsza licznik zliczający w dół, sygnalizujący potrzebę przeglądu serwisowego.

Chapter 3

Struktura programu w TIA Portal

Program zrealizowano w postaci pojedynczego bloku funkcyjnego (Function Block) FB_ZbiornikMieszajacy, wywoływanego cyklicznie w bloku organizacyjnym OB1 (Main) z powiązaną instancją danych "FB_Zbiornik Mieszajacy_DB".

3.1 Interfejs bloku

Table 3.1: Zmienne wejściowe bloku FB_ZbiornikMieszajacy

Nazwa	Typ	Opis
Start	Bool	Przycisk uruchomienia cyklu
Stop	Bool	Przycisk zatrzymania cyklu
Kasuj_Alarm	Bool	Kasowanie sygnalizacji awarii
Awaria_Zewn	Bool	Sygnał awarii z instalacji
Poziom_Analog_In	Int	Surowy sygnał z czujnika poziomu (0–27648)
Poziom_Zadany	Real	Wartość zadana poziomowi w %
Czas_Mieszania	Time	Nastawa czasu mieszania
Czas_Spustu	Time	Nastawa czasu spustu

Table 3.2: Zmienne wyjściowe bloku FB_ZbiornikMieszajacy

Nazwa	Typ	Opis
Zawor_Dozujacy_AO	Int	Wyjście analogowe sterujące zaworem
Pompa_Mieszajaca	Bool	Sterowanie pompą mieszającą
Zawor_Spustowy	Bool	Sterowanie zaworem spustowym
Lampka_Alarm	Bool	Sygnalizacja awarii
Lampka_Praca	Bool	Sygnalizacja pracy procesu
Licznik_Cykli	Int	Liczba wykonanych cykli (batchy)
Stan_Procesu	Int	Aktualny stan maszyny stanów (0–4)
Poziom_Procent	Real	Poziom przeliczony na %
Srednia_Poziomu	Real	Średnia z 10 ostatnich próbek
Serwis_Wymagany	Bool	Sygnał żądania przeglądu serwisowego

3.2 Zmienne wewnętrzne

Wewnątrz bloku zadeklarowano instancje bloków standardowych: R_TRIG, F_TRIG, TON (dwie instancje), CTU oraz CTD, a także tablicę dziesięcioelementową Historia_Poziomu : `Array[1..10] of Real` przechowującą historię próbek poziomu wykorzystywaną do obliczania wartości średniej.

Chapter 4

Omówienie kodu źródłowego

Poniżej przedstawiono realizację poszczególnych wymaganych elementów programistycznych wraz z fragmentami kodu SCL oraz odpowiadającymi zrzutami ekranu z edytora TIA Portal.

4.1 Detekcja zboczy

Wykrywanie zbocza narastającego na sygnale **Start** oraz zbocza opadającego na sygnale **Stop** zrealizowano przy użyciu standardowych bloków **R_TRIG** i **F_TRIG**:

```
1 #Start_Trigger(CLK := #Start);  
2 #Stop_Trigger(CLK := #Stop);
```

Wyjście **Start_Trigger.Q** generuje pojedynczy impuls trwający jeden cykl programu w momencie zbocza narastającego sygnału **Start**, co zabezpiecza przed wielokrotnym uruchomieniem cyklu przy przytrzymanym przycisku. Analogicznie **Stop_Trigger.Q** wykorzystywane jest do natychmiastowego, jednorazowego resetu procesu do stanu **IDLE**.

```

1
2
3 // =====
4 // CIALO BLOKU
5 // =====
6
7 // -----
8 // 8. DETEKCJA ZBOCZY - impuls narastajacy na Start, opadajacy na Stop
9 // -----
10 #Start_Trig(CLK := #Start);
11 #Stop_Trig(CLK := #Stop);
12
13 // -----
14 // 2. F. ARYTMETYCZNE - skalowanie sygnalu analogowego 0..27648 -> 0..100%
15 // -----
16 #Poziom_Procent := (INT_TO_REAL(#Poziom_Analog_In) * 100.0) / 27648.0;
17
18 // -----
19 // 4. PETLA FOR - przesuniecie probek w tablicy historii (bufor kolejkowy)
20 // -----
21 FOR #i := 10 TO 2 BY -1 DO
22   #Historia_Poziomu[#i] := #Historia_Poziomu[#i - 1];
23 END_FOR;
24 #Historia_Poziomu[1] := #Poziom_Procent;
25
26 // 4. PETLA FOR - obliczenie sredniej z 10 ostatnich probek (f. arytmetyczne + petla)
27 #Suma_Pom := 0.0;
28 FOR #i := 1 TO 10 DO
29   #Suma_Pom := #Suma_Pom + #Historia_Poziomu[#i];
30 END_FOR;
31 #Srednia_Poziomu := #Suma_Pom / 10.0;
32
33 // -----
34 // 1. F. LOGICZNE - warunki bezpieczenstwa i zezwolenia na start
35 // -----
36 #Blad_Awarii := #Awaria_Zewn OR (#Poziom_Procent > #Poziom_Max);
37
38 #Zezwolenie_Start := NOT #Blad_Awarii
39 AND (#Poziom_Procent < #Poziom_Max)
40 AND NOT #Stop;
41
42 IF #Kasuj_Alarm THEN

```

Figure 4.1: Fragment kodu — detekcja zboczy, skalowanie sygnału analogowego oraz pętle FOR

4.2 Funkcje arytmetyczne

Skalowanie sygnału z modułu wejść analogowych (zakres 0–27648) do wartości procentowej realizuje poniższa instrukcja:

```

1 #Poziom_Procent := (INT_TO_REAL(#Poziom_Analog_In) * 100.0) / 27648.0;

```

Dodatkowo funkcje arytmetyczne wykorzystano przy obliczaniu uchybu regulacji (różnica poziomu zadanego i rzeczywistego), przeliczaniu procentowego otwarcia zaworu na wartość surową sygnału AO oraz przy sumowaniu próbek historii w celu wyznaczenia wartości średniej.

4.3 Pętle FOR

Zaimplementowano dwie pętle FOR. Pierwsza realizuje przesunięcie próbek w buforze kołowym (najstarsza próbka jest usuwana, nowa wstawiana na początek tablicy):

```

1 FOR #i := 10 DOWNT0 2 DO
2   #Historia_Poziomu[#i] := #Historia_Poziomu[#i - 1];

```



```

3 END_FOR;
4 #Historia_Poziomu[1] := #Poziom_Procent;

```

Druuga pętla sumuje zawartość tablicy w celu obliczenia średniej kroczącej z 10 ostatnich pomiarów:

```

1 #Suma_Pom := 0.0;
2 FOR #i := 1 TO 10 DO
3     #Suma_Pom := #Suma_Pom + #Historia_Poziomu[#i];
4 END_FOR;
5 #Srednia_Poziomu := #Suma_Pom / 10.0;

```

4.4 Funkcje logiczne

Warunki bezpieczeństwa oraz zezwolenia na start cyklu zbudowano z operatorów logicznych AND, OR, NOT:

```

1 #Blad_Awarii := #Awaria_Zewn OR (#Poziom_Procent > #Poziom_Max);
2
3 #Zezwolenie_Start := NOT #Blad_Awarii
4                     AND (#Poziom_Procent < #Poziom_Max)
5                     AND NOT #Stop;

```

```

85     #Otwarcie_Procent := 0.0;
86 END_IF;
87
88 #Zawor_Dozujacy_AO := REAL_TO_INT((#Otwarcie_Procent * 27648.0) / 100.0);
89
90 IF #Poziom_Procent >= #Poziom_Zadany THEN
91     #Zawor_Dozujacy_AO := 0;
92     #Stan_Procesu := 2;
93 END_IF;
94
95 IF #Blad_Awarii THEN
96     #Zawor_Dozujacy_AO := 0;
97     #Stan_Procesu := 0;
98 END_IF;
99
100 2: // MIESZANIE - 5. TIMER TON
101     #Pompa_Mieszajaca := TRUE;
102     #Timer_Mieszania(IN := TRUE,
103                     PT := #Czas_Mieszania);
104
105 IF #Timer_Mieszania.Q THEN
106     #Timer_Mieszania(IN := FALSE,
107                     PT := #Czas_Mieszania);
108     #Pompa_Mieszajaca := FALSE;
109     #Stan_Procesu := 3;
110 END_IF;
111
112 IF #Blad_Awarii THEN
113     #Pompa_Mieszajaca := FALSE;
114     #Timer_Mieszania(IN := FALSE,
115                     PT := #Czas_Mieszania);
116     #Stan_Procesu := 0;
117 END_IF;
118
119 3: // SPUST - 5. TIMER TON
120     #Zawor_Spustowy := TRUE;
121     #Timer_Spustu(IN := TRUE,
122                  PT := #Czas_Spustu);
123
124 IF #Timer_Spustu.Q THEN

```

Figure 4.2: Fragment kodu — funkcje logiczne, obsługa zbrocza Stop oraz początek maszyny stanów CASE

4.5 Instrukcje warunkowe — maszyna stanów

Główną logikę sterującą procesem zrealizowano w postaci instrukcji CASE, w której każdy stan odpowiada jednemu etapowi procesu technologicznego (IDLE, NAPEŁNIANIE, MIESZANIE, SPUST, ZAKOŃCZONO). Dodatkowo w obrębie poszczególnych gałęzi zastosowano instrukcje IF...THEN...ELSE do regulacji proporcjonalnej otwarcia zaworu oraz ograniczenia zakresu sterowania do 0–100%.

```

43 #Blad_Awarii := FALSE;
44 END_IF;
45
46 #Lampka_Alarm := #Blad_Awarii;
47
48 // reakcja na zbocze opadające Stop -> natychmiastowy powrót do IDLE
49 IF #Stop_Trig.Q THEN
50   #Stan_Procesu := 0;
51   #Pompa_Mieszajaca := FALSE;
52   #Zawor_Spustowy := FALSE;
53   #Zawor_Dozujacy_AO := 0;
54 END_IF;
55
56 // -----
57 // 3. INSTRUKCJE WARUNKOWE - maszyna stanow procesu (CASE)
58 // -----
59 CASE #Stan_Procesu OF
60
61   0: // IDLE - oczekiwanie na start
62     #Lampka_Praca := FALSE;
63     #Pompa_Mieszajaca := FALSE;
64     #Zawor_Spustowy := FALSE;
65     #Zawor_Dozujacy_AO := 0;
66
67     IF #Start_Trig.Q AND #Zezwolenie_Start THEN
68       #Stan_Procesu := 1;
69     END_IF;
70
71   1: // NAPELNIANIE - 7. ZAWOR ANALOGOWY, regulacja proporcjonalna (P)
72     #Lampka_Praca := TRUE;
73     #Blad_Regulacji := #Poziom_Zadany - #Poziom_Procent;
74
75     IF #Blad_Regulacji > 0.0 THEN
76       #Otwarcie_Procent := #Blad_Regulacji * #Wzmocnienie;
77     ELSE
78       #Otwarcie_Procent := 0.0;
79     END_IF;
80
81     // ograniczenie zakresu 0-100%
82     IF #Otwarcie_Procent > 100.0 THEN
83       #Otwarcie_Procent := 100.0;
84     ELSIF #Otwarcie_Procent < 0.0 THEN

```

Figure 4.3: Fragment kodu — regulacja proporcjonalna zaworu, stan MIESZANIE i SPUST (timery TON)

4.6 Timery

Do odmierzania czasu mieszania oraz czasu spustu wykorzystano dwie niezależne instancje bloku opóźnienia załączenia TON:

```

1 #Timer_Mieszania(IN := TRUE, PT := #Czas_Mieszania);
2 IF #Timer_Mieszania.Q THEN
3   ...
4 END_IF;

```

4.7 Liczniki

Zliczanie wykonanych cykli technologicznych (batchy) zrealizowano licznikiem zliczającym w górę CTU, natomiast odliczanie do przeglądu serwisowego — licznikiem zliczającym w dół CTD, kasowanym sygnałem Kasuj_Alarm:

```

1 #Licznik_Batch(CU := TRUE, R := FALSE, PV := 9999);
2 #Licznik_Cykli := #Licznik_Batch.CV;
3
4 #Licznik_Serwis(CD := #Blad_Awarii, LD := #Kasuj_Alarm, PV := 5);
5 #Serwis_Wymagany := #Licznik_Serwis.Q;

125 #Timer_Spustu(IN := FALSE,
126             PT := #Czas_Spustu);
127 #Zawor_Spustowy := FALSE;
128 #Stan_Procesu := 4;
129 END_IF;
130
131 4: // ZAKONCZONO - 6. LICZNIK CTU zlicza wykonany cykl (batch)
132 #Licznik_Batch(CU := TRUE,
133             R := FALSE,
134             PV := 9999);
135 #Licznik_Cykli := #Licznik_Batch.CV;
136 #Lampka_Praca := FALSE;
137 #Stan_Procesu := 0;
138
139 ELSE
140     #Stan_Procesu := 0;
141
142 END_CASE;
143
144 // -----
145 // 6. LICZNIK CTD - odliczanie do przeglądu serwisowego
146 // (kazda awaria zmniejsza licznik, reset po przeglądzie/kasowaniu)
147 // -----
148 #Licznik_Serwis(CD := #Blad_Awarii,
149             LD := #Kasuj_Alarm,
150             PV := 5);
151 #Serwis_Wymagany := #Licznik_serwis.Q;
152
153

```

Figure 4.4: Fragment kodu — stan ZAKOŃCZONO (licznik CTU) oraz licznik serwisowy CTD

4.8 Sterowanie zaworem analogowym

W stanie NAPEŁNIANIE zaimplementowano prosty regulator proporcjonalny (P), którego wyjście po ograniczeniu do zakresu 0–100% jest przeliczane na wartość surową sygnału wyjściowego AO:

```

1 #Blad_Regulacji := #Poziom_Zadany - #Poziom_Procent;
2

```

```
3 IF #Blad_Regulacji > 0.0 THEN
4     #Otwarcie_Procent := #Blad_Regulacji * #Wzmocnienie;
5 ELSE
6     #Otwarcie_Procent := 0.0;
7 END_IF;
8
9 IF #Otwarcie_Procent > 100.0 THEN
10     #Otwarcie_Procent := 100.0;
11 ELSIF #Otwarcie_Procent < 0.0 THEN
12     #Otwarcie_Procent := 0.0;
13 END_IF;
14
15 #Zawor_Dozujacy_A0 := REAL_TO_INT((#Otwarcie_Procent * 27648.0) / 100.0)
    ;
```

Chapter 5

Uruchomienie i testowanie programu

5.1 Metodyka testów

Program został przetestowany bez użycia fizycznego sterownika, z wykorzystaniem symulatora **PLCSIM** zintegrowanego z TIA Portal. Weryfikację poprawności działania przeprowadzono poprzez podgląd w trybie monitorowania (*monitoring mode*) bezpośrednio na graficznym symbolu wywołania bloku funkcyjnego w OB1, gdzie w czasie rzeczywistym widoczne są wartości wszystkich zmiennych wejściowych i wyjściowych instancji "FB_Zbiornik Mieszajacy_DB". Wartości sygnałów wejściowych (poziom, wartość zadana, sygnały Start/Stop) wymuszano ręcznie w celu weryfikacji poprawności przejść pomiędzy poszczególnymi stanami procesu.

5.2 Scenariusze testowe i wyniki

Test 1 — stan spoczynkowy (IDLE)

Przy braku wymuszenia sygnału Start oraz zerowym poziomie w zbiorniku (**Poziom_Analog_In** = 0) program pozostaje w stanie **Stan_Procesu** = 0. Wszystkie wyjścia sterujące (pompa, zawór spustowy, zawór dozujący, lampka pracy) są nieaktywne, co potwierdza poprawną inicjalizację i logikę stanu IDLE.

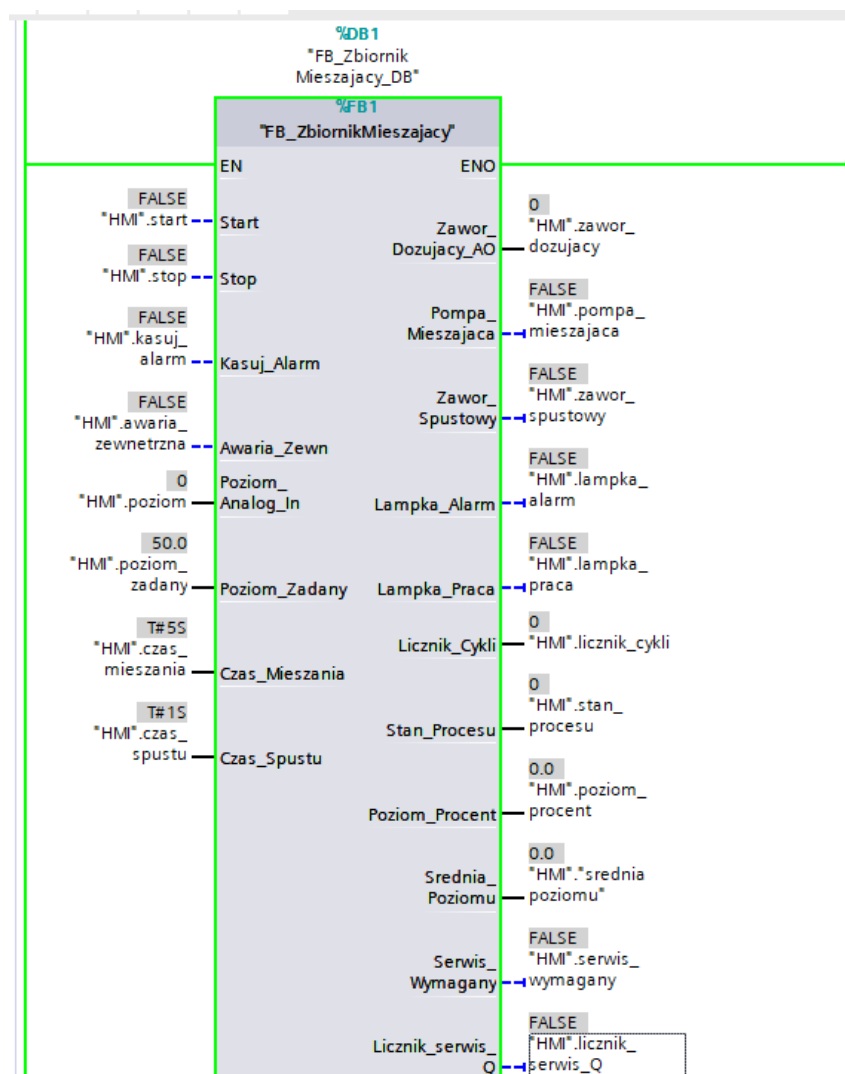


Figure 5.1: Monitoring instancji bloku — stan spoczynkowy IDLE (Stan_Procesu = 0)

Test 2 — przejście do stanu NAPEŁNIANIE

Po wymuszeniu impulsu na wejściu **Start** program przeszedł do stanu **Stan_Procesu = 1** (NAPEŁNIANIE). Przy dużym uchybie regulacji (poziom zadany 50%, poziom rzeczywisty 0%) regulator proporcjonalny wysterował zawór dozujący do wartości maksymalnej — **Zawor_Dozejacy_AO = 27648** (100% otwarcia), co potwierdza poprawne działanie ograniczenia zakresu regulacji (*clamping*) oraz sygnalizacji pracy (**Lampka_Praca = TRUE**).

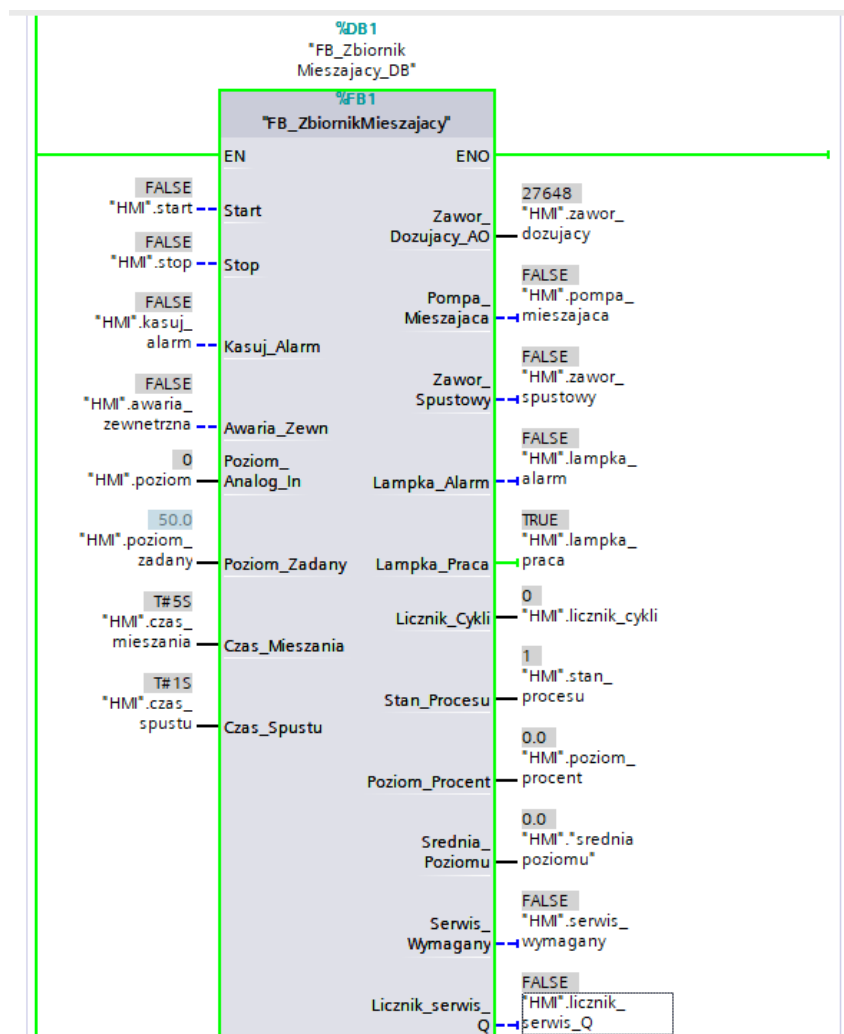


Figure 5.2: Monitoring instancji bloku — stan NAPEŁNIANIE, pełne otwarcie zaworu przy dużym uchybie

Test 3 — weryfikacja skalowania i uśredniania poziomu

Przy wymuszeniu sygnału `Poziom_Analog_In = 6000` program poprawnie przeliczył wartość na jednostki procentowe zgodnie ze wzorem skalującym:

$$Poziom_Procent = \frac{6000 \cdot 100}{27648} \approx 21,70\%$$

co zostało potwierdzone odczytem `Poziom_Procent = 21.70139` widocznym na zrzucie ekranu. Wartość `Srednia_Poziomu` przyjęła identyczną wartość, co jest zgodne z oczekiwaniami przy pierwszym cyklu wypełniania tablicy historii (wszystkie 10 elementów bufora zawiera jeszcze wartości domyślne z tej samej próbki).

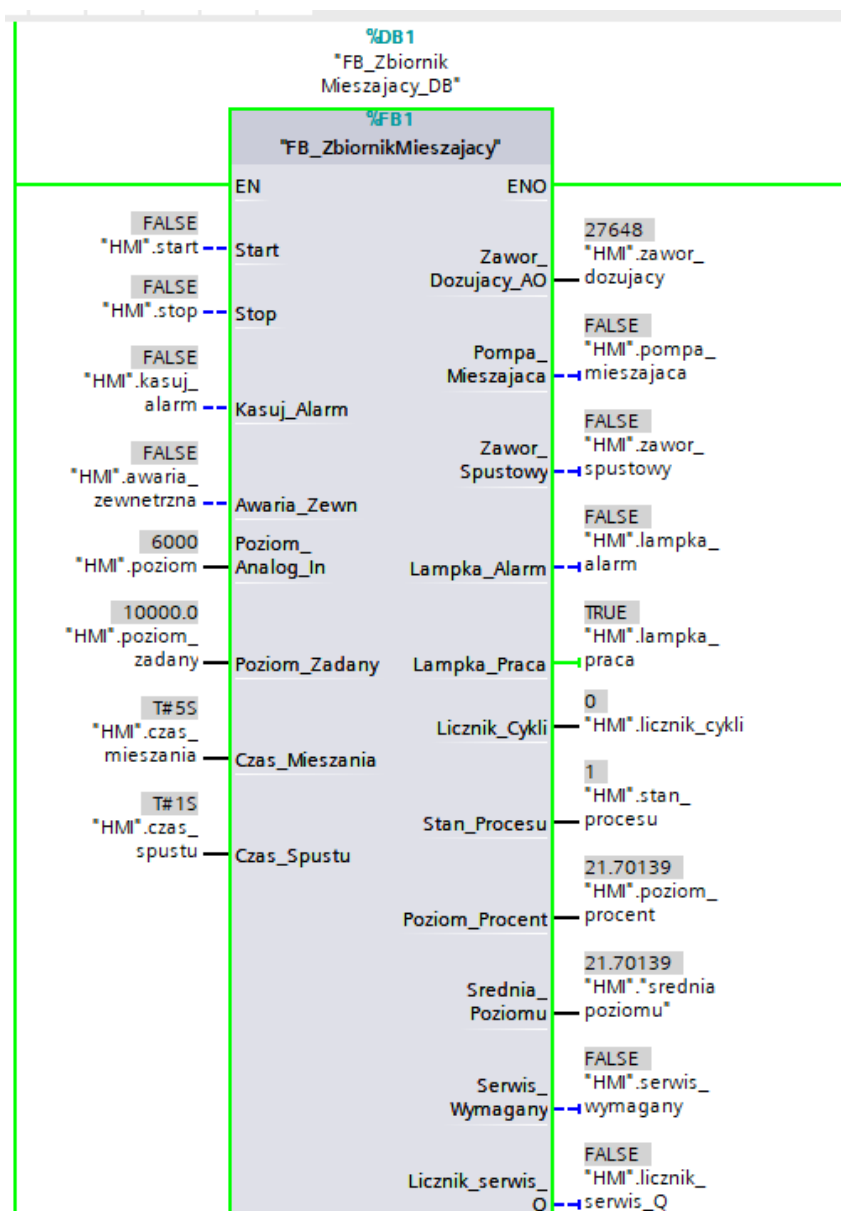


Figure 5.3: Monitoring instancji bloku — poprawne przeliczenie sygnału analogowego na wartość procentową

Test 4 — powrót do stanu IDLE

Po wymuszeniu sygnału Stop (zbrocze opadające) program natychmiast wrócił do stanu `Stan_Procesu = 0`, wyzerował wyjście zaworu dozującego oraz wyłączył sygnalizację pracy, co potwierdza poprawność obsługi przerwania cyklu w dowolnym jego momencie.

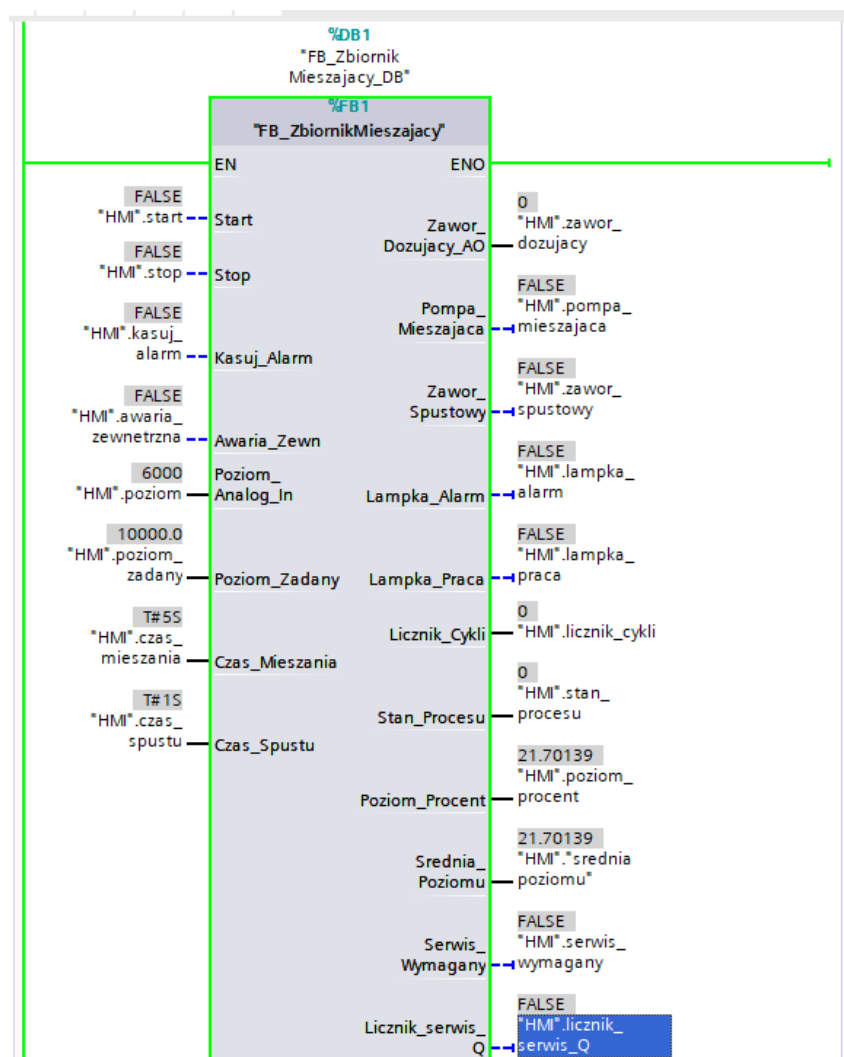


Figure 5.4: Monitoring instancji bloku — powrót do stanu IDLE po zboczu opadającym sygnału Stop

5.3 Uwagi z testów

Podczas testów zaobserwowano, że w niektórych przebiegach testowych parametr **Poziom_Zadany** był wymuszany wartością znacznie przekraczającą zakres 0–100 (np. 10000.0), co nie odpowiada założeniu przyjętemu w projekcie (wartość zadana powinna być podawana w jednostkach procentowych). W takim przypadku uchyb regulacji jest zawsze bardzo duży, a regulator proporcjonalny utrzymuje zawór w pełnym otwarciu niezależnie od aktualnego poziomu — jest to zachowanie zgodne z zaimplementowanym ograniczeniem zakresu sterowania (*clamping do 100%*), a nie błąd programu. Wniosek ten wskazuje na konieczność dodania w przyszłej wersji programu kontroli poprawności zakresu wartości zadawanych przez operatora (walidacja wejścia **Poziom_Zadany** do przedziału 0–100).

Chapter 6

Wnioski

1. Zaimplementowany program w pełni realizuje wszystkie założone wymagania projektowe: funkcje logiczne, funkcje arytmetyczne, instrukcje warunkowe, pętle, timery, liczniki, sterowanie zaworem analogowym oraz detekcję zboczy sygnałów.
2. Zastosowanie maszyny stanów opartej na instrukcji CASE pozwoliło na czytelne i łatwe w rozbudowie zaprogramowanie sekwencji technologicznej.
3. Testy przeprowadzone w środowisku symulacyjnym PLCSIM z podglądem online na bloku wywołania potwierdziły poprawność działania wszystkich kluczowych mechanizmów programu — skalowania sygnału analogowego, regulacji proporcjonalnej zaworu, przejść między stanami procesu oraz reakcji na sygnały Start/Stop.
4. Zidentyfikowano potencjalny obszar do rozwoju programu — walidację zakresu wartości zadawanych przez operatora, co zwiększyłoby odporność aplikacji na błędne dane wejściowe.
5. W dalszym rozwoju projektu warto rozważyć zastąpienie regulatora proporcjonalnego (P) regulatorem PI w celu eliminacji uchybu ustalonego oraz dodanie wizualizacji HMI umożliwiającej operatorowi łatwiejsze zadawanie parametrów procesu.

Appendix A

Pełny kod źródłowy

A.1 Blok funkcyjny FB_ZbiornikMieszajacy

```
1 // =====
2 // PROJEKT: Automatyczny zbiornik mieszajacy (dozowanie / mieszanie /
3 // spust)
4 // TIA Portal - język SCL
5 // Blok: FB_ZbiornikMieszajacy
6 //
7 // Zawiera:
8 // 1. Funkcje logiczne      -> AND / OR / NOT (warunki
9 //    bezpieczeństwa)
10 // 2. Funkcje arytmetyczne  -> skalowanie AI, regulacja
11 //    proporcjonalna
12 // 3. Instrukcje warunkowe -> IF..THEN..ELSE, CASE (maszyna stanów)
13 // 4. Petle                 -> FOR (przesuwanie tablicy historii,
14 //    srednia)
15 // 5. Timery                -> TON (czas mieszania, czas spustu)
16 // 6. Liczniki              -> CTU (licznik cykli), CTD (licznik
17 //    serwisowy)
18 // 7. Zawor analogowy      -> wyjscie AO sterowane proporcjonalnie
19 // 8. Detekcja zboczy       -> R_TRIG (Start), F_TRIG (Stop)
20 // =====
21
22 FUNCTION_BLOCK "FB_ZbiornikMieszajacy"
23 { S7_Optimized_Access := 'TRUE' }
24
25 VERSION : 0.1
26
27 VAR_INPUT
28
29     Start          : Bool;           // przycisk Start
30     Stop           : Bool;           // przycisk Stop
31     Kasuj_Alarm    : Bool;           // reset alarmu
32     Awaria_Zewn    : Bool;           // sygnal awarii z
33         instalacji (np. brak zasilania mieszkadla)
34     Poziom_Analog_In : Int;           // surowy sygnal AI z
35         czujnika poziomu (0..27648)
36     Poziom_Zadany   : Real;           // wartosc zadana
37         poziomu w % (0..100)
38     Czas_Mieszania  : Time := T#10s; // nastawa czasu
39         mieszania
40     Czas_Spustu     : Time := T#8s;   // nastawa czasu spustu
41
42 END_VAR
```

```

32 VAR_OUTPUT
33     Zawor_Dozujacy_A0      : Int;                // wyjście analogowe do
           zaworu dozujacego (0..27648)
34     Pompa_Mieszajaca      : Bool;
35     Zawor_Spustowy         : Bool;
36     Lampka_Alarm           : Bool;
37     Lampka_Praca           : Bool;
38     Licznik_Cykli          : Int;                // liczba wykonanych
           batchow
39     Stan_Procesu           : Int;                // 0-Idle,1-Napelnianie
           ,2-Mieszanie,3-Spust,4-Zakonczono
40     Poziom_Procent         : Real;              // poziom przeliczony na
           %
41     Srednia_Poziomu        : Real;              // srednia z ostatnich
           10 probek
42     Serwis_Wymagany        : Bool;              // sygnał do przeglądu
           po serii awarii
43 END_VAR
44
45 VAR
46     // ---- 8. Detekcja zboczy ----
47     Start_Trig             : R_TRIG;
48     Stop_Trig              : F_TRIG;
49
50     // ---- 5. Timery ----
51     Timer_Mieszania        : TON;
52     Timer_Spustu           : TON;
53
54     // ---- 6. Liczniki ----
55     Licznik_Batch          : CTU;
56     Licznik_Serwis         : CTD;
57
58     // ---- pomocnicze do logiki / regulacji ----
59     Zezwolenie_Start       : Bool;
60     Blad_Awarri            : Bool;
61     Blad_Regulacji         : Real;
62     Wzmocnienie            : Real := 5.0;        // wzmocnienie
           regulatora P
63     Otwarcie_Procent       : Real;
64
65     // ---- 4. tablica do petli / historia poziomu ----
66     Historia_Poziomu        : Array[1..10] of Real;
67     i                       : Int;
68     Suma_Pom               : Real;
69
70     Poziom_Max              : Real := 95.0;      // % - alarm
           przepelnienia
71     Poziom_Min             : Real := 5.0;        // % - poziom minimalny
72 END_VAR
73
74 // =====
75 // CIALO BLOKU
76 // =====
77
78 // -----
79 // 8. DETEKCJA ZBOCZY - impuls narastajacy na Start, opadajacy na Stop
80 // -----
81 #Start_Trig(CLK := #Start);

```

```

82 #Stop_Trig(CLK := #Stop);
83
84 // -----
85 // 2. F. ARYTMETYCZNE - skalowanie sygnału analogowego 0..27648 ->
86 // 0..100%
87 // -----
88 #Poziom_Procent := (Int_To_Real(#Poziom_Analog_In) * 100.0) / 27648.0;
89
90 // -----
91 // 4. PETLA FOR - przesunięcie probek w tablicy historii (bufor
92 // kolejkowy)
93 // -----
94 FOR #i := 10 DOWNTO 2 DO
95     #Historia_Poziomu[#i] := #Historia_Poziomu[#i - 1];
96 END_FOR;
97 #Historia_Poziomu[1] := #Poziom_Procent;
98
99 // 4. PETLA FOR - obliczenie średniej z 10 ostatnich probek (f.
100 // arytmetyczne + petla)
101 #Suma_Pom := 0.0;
102 FOR #i := 1 TO 10 DO
103     #Suma_Pom := #Suma_Pom + #Historia_Poziomu[#i];
104 END_FOR;
105 #Srednia_Poziomu := #Suma_Pom / 10.0;
106
107 // -----
108 // 1. F. LOGICZNE - warunki bezpieczeństwa i zezwolenia na start
109 // -----
110 #Blad_Awarii := #Awaria_Zewn OR (#Poziom_Procent > #Poziom_Max);
111
112 #Zezwolenie_Start := NOT #Blad_Awarii
113                     AND (#Poziom_Procent < #Poziom_Max)
114                     AND NOT #Stop;
115
116 IF #Kasuj_Alarm THEN
117     #Blad_Awarii := FALSE;
118 END_IF;
119
120 #Lampka_Alarm := #Blad_Awarii;
121
122 // reakcja na zbocze opadające Stop -> natychmiastowy powrót do IDLE
123 IF #Stop_Trig.Q THEN
124     #Stan_Procesu      := 0;
125     #Pompa_Mieszajaca := FALSE;
126     #Zawor_Spustowy   := FALSE;
127     #Zawor_Dozujacy_A0 := 0;
128 END_IF;
129
130 // -----
131 // 3. INSTRUKCJE WARUNKOWE - maszyna stanów procesu (CASE)
132 // -----
133 CASE #Stan_Procesu OF
134
135     0: // IDLE - oczekiwanie na start
136         #Lampka_Praca      := FALSE;
137         #Pompa_Mieszajaca := FALSE;
138         #Zawor_Spustowy   := FALSE;
139         #Zawor_Dozujacy_A0 := 0;

```

```
137
138     IF #Start_Trig.Q AND #Zezwolenie_Start THEN
139         #Stan_Procesu := 1;
140     END_IF;
141
142 1:  // NAPELNIANIE - 7. ZAWOR ANALOGOWY, regulacja proporcjonalna (P
    )
143     #Lampka_Praca := TRUE;
144     #Blad_Regulacji := #Poziom_Zadany - #Poziom_Procent;
145
146     IF #Blad_Regulacji > 0.0 THEN
147         #Otwarcie_Procent := #Blad_Regulacji * #Wzmocnienie;
148     ELSE
149         #Otwarcie_Procent := 0.0;
150     END_IF;
151
152     // ograniczenie zakresu 0-100%
153     IF #Otwarcie_Procent > 100.0 THEN
154         #Otwarcie_Procent := 100.0;
155     ELSIF #Otwarcie_Procent < 0.0 THEN
156         #Otwarcie_Procent := 0.0;
157     END_IF;
158
159     #Zawor_Dozujacy_A0 := Real_To_Int((#Otwarcie_Procent * 27648.0)
        / 100.0);
160
161     IF #Poziom_Procent >= #Poziom_Zadany THEN
162         #Zawor_Dozujacy_A0 := 0;
163         #Stan_Procesu := 2;
164     END_IF;
165
166     IF #Blad_Awarii THEN
167         #Zawor_Dozujacy_A0 := 0;
168         #Stan_Procesu := 0;
169     END_IF;
170
171 2:  // MIESZANIE - 5. TIMER TON
172     #Pompa_Mieszajaca := TRUE;
173     #Timer_Mieszania(IN := TRUE, PT := #Czas_Mieszania);
174
175     IF #Timer_Mieszania.Q THEN
176         #Timer_Mieszania(IN := FALSE, PT := #Czas_Mieszania);
177         #Pompa_Mieszajaca := FALSE;
178         #Stan_Procesu := 3;
179     END_IF;
180
181     IF #Blad_Awarii THEN
182         #Pompa_Mieszajaca := FALSE;
183         #Timer_Mieszania(IN := FALSE, PT := #Czas_Mieszania);
184         #Stan_Procesu := 0;
185     END_IF;
186
187 3:  // SPUST - 5. TIMER TON
188     #Zawor_Spustowy := TRUE;
189     #Timer_Spustu(IN := TRUE, PT := #Czas_Spustu);
190
191     IF #Timer_Spustu.Q THEN
192         #Timer_Spustu(IN := FALSE, PT := #Czas_Spustu);
```

```

193         #Zawor_Spustowy := FALSE;
194         #Stan_Procesu := 4;
195     END_IF;
196
197     4: // ZAKONCZONO - 6. LICZNIK CTU zlicza wykonany cykl (batch)
198         #Licznik_Batch(CU := TRUE, R := FALSE, PV := 9999);
199         #Licznik_Cykli := #Licznik_Batch.CV;
200         #Lampka_Praca := FALSE;
201         #Stan_Procesu := 0;
202
203     ELSE
204         #Stan_Procesu := 0;
205
206 END_CASE;
207
208 // -----
209 // 6. LICZNIK CTD - odliczanie do przeglądu serwisowego
210 // (każda awaria zmniejsza licznik, reset po przeglądzie/kasowaniu)
211 // -----
212 #Licznik_Serwis(CD := #Blad_Awarii, LD := #Kasuj_Alarm, PV := 5);
213 #Serwis_Wymagany := #Licznik_Serwis.Q;
214
215 END_FUNCTION_BLOCK

```

A.2 Wywołanie w OB1 (Main)

```

1 // =====
2 // Przykład wywołania FB_ZbiornikMieszajacy w OB1 (Main)
3 // Wklej poniższy fragment do istniejącego bloku "Main [OB1]" w SCL.
4 // Wcześniej należy utworzyć instancje DB: "DB_ZbiornikMieszajacy"
5 // jako instancje bloku FB_ZbiornikMieszajacy.
6 // =====
7
8 "DB_ZbiornikMieszajacy"(
9     // --- wejścia (podłącz realne adresy I / sygnały HMI) ---
10    Start      := "HMI_Start",           // Bool - przycisk/bit
        ze zdalnej wizualizacji
11    Stop       := "HMI_Stop",           // Bool
12    Kasuj_Alarm := "HMI_Kasuj_Alarm",    // Bool
13    Awaria_Zewn := "IW_Awaria_Mieszadla", // Bool - np. termik
        silnika
14    Poziom_Analog_In := "IW100",         // Int - wejście
        analogowe (czujnik poziomu)
15    Poziom_Zadany := "HMI_Poziom_Zadany", // Real - nastawa z
        panelu operatorskiego
16    Czas_Mieszania := T#15s,            // Time - można też
        podpiąć zmienna z HMI
17    Czas_Spustu := T#10s,
18
19    // --- wyjścia (podłącz realne adresy Q) ---
20    Zawor_Dożujacy_AO => "QW100",        // Int -> modul AO
        sterujący zaworem
21    Pompa_Mieszajaca => "Q0.0",
22    Zawor_Spustowy   => "Q0.1",
23    Lampka_Alarm     => "Q0.2",
24    Lampka_Praca     => "Q0.3",

```

```
25   Licznik_Cykli      => "HMI_Licznik_Cykli" ,  
26   Stan_Procesu      => "HMI_Stan_Procesu" ,  
27   Poziom_Procent    => "HMI_Poziom_Procent" ,  
28   Srednia_Poziomu   => "HMI_Srednia_Poziomu" ,  
29   Serwis_Wymagany   => "HMI_Serwis_Wymagany"  
30 );
```